

HelixMemory

HelixMemory

AI ajanları için tek bir hafıza beyni — dört üstün performanslı motor, birleştirilmiş.

Özet

HelixMemory, dört önde gelen hafıza sistemini (Mem0, Cognee, Letta, Graphiti) tek bir bilişsel hafıza motorunda birleştiren bir Go SDK'dır. Bu motor, bu sistemleri paralel olarak tarar ve sonuçları birleştirir. AI uygulamalarına, dört ayrı hafıza katmanı yerine tek bir dayanıklı, tekrarsız ve yeniden sıralanmış hafıza katmanı sunar.

Kısa açıklama

HelixMemory, Mem0, Cognee, Letta ve Graphiti'ü AI uygulamaları için tek bir birleşik bilişsel hafıza motorunda birleştiren bir Go SDK'dır. Yazma işlemlerini akıllıca yönlendirir, tüm arka uçları paralel olarak tarar ve sonuçları üç aşamalı bir toplama-tekillleştirme-yeniden sıralama hattı aracılığıyla birleştirir.

Ayrıntılı açıklama

HelixMemory, AI uygulamaları için birleşik bir bilişsel hafıza motorudur ve bir Go SDK (modül `digital.vasic.helixmemory`, Go 1.25+) olarak sunulur. Temel varsayımı, hiçbir hafıza projesinin her alanda en iyi olamayacağıdır — bu nedenle, sıfırdan hafıza yeniden uygulamak ve tek bir projenin kör noktalarını devralmak yerine, dört üstün sistemin her birinin güçlü yönlerinden yararlanmasını sağlar: dinamik bilgi çıkarma ve tercih yönetimi için Mem0, ECL boru hatlarıyla oluşturulan anlamsal bilgi grafikleri için Cognee, düzenlenebilir hafıza blokları ve uyku

zamanı hesaplamalarıyla durum bilgisi taşıyan bir ajan çalışma zamanı için Letta, ve zaman içinde bilgilerin nasıl değiştiğini değerlendiren iki zamanlı bir bilgi grafiği için Graphiti.

Bu dört bağımsız depoyu tek bir beyne dönüştüren şey, birleştirme motorudur. Yazma yolunda, gelen her hafıza içeriğine göre sınıflandırılır ve en uygun arka uca yönlendirilir. Okuma yolunda ise sorgu, tüm arka uçlara paralel olarak dağıtılır ve ham sonuçlar üç aşamalı bir birleştirme hattına —toplama, tekilleştirme ve ardından kaynaklar arası yeniden sıralama— akar. Böylece çağırın taraf, dört gürültülü ve örtüşen sonuç kümesi yerine yalnızca tek bir temiz, sıralanmış yanıt görür. Her arka uç, sorunsuz bir şekilde devre dışı bırakılabilmesi için devre kesicilerle sarılmıştır: Bir motor çöktüğünde, devre kesici devreye girer ve kalan arka uçlar, tüm hafıza katmanını beraberinde sürüklemekten hizmet vermeye devam eder. Motor, doğrudan bir hafıza sağlayıcısının yerine geçebilecek şekilde MemoryStore arayüzünü uyguladığı için, çağırın tarafın mimarisini değiştirmeye gerek kalmaz. Prometheus metrikleri ise yönlendirme ve birleştirme süreçlerinin iç işleyişini tamamen gözlemlenebilir hale getirir.

HelixMemory, daha geniş Helix AI topluluğu olan HelixAgent için hafıza katmanı olarak geliştirilmiştir ve ailenin blöf karşıtı test disiplini hafıza alanına taşır: Dahili bir meydan okuma çalıştırıcısı, yönlendirme, birleştirme, çevirici ve devre kesici gibi gerçek üretim kod yollarını çalıştırırken, eşlenik mutasyon sarmalayıcısı kasıtlı olarak değişmezleri bozarak testlerin, mantık hatalı olduğunda gerçekten başarısız olduğunu kanıtlar. Bu sayede, yeşil bir test paketi gerçekten bir anlam ifade eder.

İçerik

AI ajanlarının uzun ömürlü, yüksek kaliteli belleğe ihtiyacı var, ancak ekosistem parçalanmış durumda — her bellek projesi (Mem0, Cognee, Letta, Graphiti) bir konuda güçlü, diğerlerinde zayıf. HelixMemory, HelixAgent için bu sistemlerin güçlü yönlerini tek bir bellek yüzeyinde birleştirmek, herhangi birine bağımlılık yaratmadan sunmak üzere tasarlandı.

Neden oyunun kurallarını değiştiriyor?

Zorunlu seçimi ortadan kaldırıyor. Normalde aynı yuva için rekabet eden dört bellek sistemi, tek bir arayüzün arkasında tamamlayıcı arka uçlara dönüşüyor — böylece bir uygulama, dinamik gerçek çıkarma, anlamsal bilgi grafikleri, durum tabanlı ajan belleği ve çift zamanlı akıl yürütmeyi *eş zamanlı olarak* elde ediyor; yinelenen verilerin ayıklanması ve kaynaklar arası yeniden sıralama otomatik olarak hallediliyor. Daha önce pratik olmayan şey, “hangi bellek motorunu benimsemeliyiz?” sorusunu yanlış bir ikilem olarak görmek: HelixMemory, tek bir MemoryStore arayüzü

arkasında tüm bu sistemlerin güçlü yönlerine sahip olmanızı sağlıyor, tek bir motorun kör noktalarını devralmadan ya da bağımlılık yaratmadan.

Yenilikçi yönleri

- **Çoklu arka uç füzyonu** (topla → yinelenenleri ayıkla → kaynaklar arası yeniden sırala): Çağrıyı tek bir depoya bağlamak yerine, tek bir sıralanmış sonuç kümesi döndürür.
- **Akıllı yazma yönlendirmesi**: Her bellek içeriğine göre sınıflandırılır ve en uygun depoya gönderilir; böylece doğru veri doğru depoda yer alır.
- **Arka uç başına devre kesicilerle zarif bozulma**: Başarısız olan bir motor izole edilir, sistem çökmez ve diğerleri hizmet vermeye devam eder.
- **Uyku zamanında hesaplama birleştirmesi** (Letta aracılığıyla): Bellek, yalnızca sorgu anında değil, boşta kalınan sürelerde yeniden düzenlenir.
- **Sahtekârlık karşıtı doğrulama**: Gerçek üretim koduyla çalışan bir meydan okuma yürütücüsü, bir değişmezlik tersine çevrildiğinde başarısız olması gereken bir mutasyon sarmalayıcısıyla eşleştirilir — böylece test kapısının gerçek bir kontrol olduğu, totolojiden ibaret olmadığı kanıtlanır.

En büyük teknik zorluklar ve çözümleri

- **Dört farklı arka ucu tek bir tutarlı sonuç kümesinde birleştirmek** — Her motor belleği kendi biçiminde döndürür ve bunları naif bir şekilde birleştirmek yinelenen sonuçlar ve karşılaştırılamaz sıralamalar üretir. Çözüm: Kaynaklar arasında toplayan, örtüşmeleri ayıklayan ve her şeyi ortak bir zeminde yeniden sıralayan, birleştirilmiş sayım değişmezliğini testlerde doğrulayan tip güvenli bir füzyon motoru.
- **Bir arka uç çöktüğünde sistemin ayakta kalması** — Ulaşılamayan bir bellek motoru tüm katmanı durduramaz. Çözüm: Her arka uç için devre kesiciler, kapalı → açık (başarısızlık eşiğinden sonra) → yarı açık (zaman aşımından sonra) durum makinesi kullanarak sorunlu arka ucu izole eder ve sistem sağlıklı olanlardan hizmet vermeye devam eder, ta ki sorunlu olan düzelene kadar.
- **Bellek mantığının yalnızca derlenmekle kalmayıp gerçekten çalıştığını kanıtlamak** — Yeşil bir test paketi, testlerin başarısız olamayacağı durumda anlamsızdır. Çözüm: Gerçek üretim kodunu (yönlendirme, füzyon, çevirici, devre kesici) çalıştıran bir süreç içi meydan okuma yürütücüsü ve değişmezlikleri tersine çevirerek testlerin kırmızıya dönmesini zorunlu kılan eşleştirilmiş bir mutasyon sarmalayıcısı — böylece kontrol kapısının totoloji olmadığı doğrulanır.

Teknoloji Yığını

- **Go (1.25+)** — Tek SDK ve çalışma zamanı; dört arka uç arasında paralel okuma dağıtımı bir eşzamanlılık sorunu olduğundan ve Go'un goroutine'leri bu işlemi düşük maliyetli kıldığından, ayrıca arayüz türleriyle tüm sistemin tek bir temiz dikiş noktası (MemoryStore) sunduğundan tercih edildi.
- **Mem0** — Dinamik olgu çıkarımı ve tercih yönetimi arka ucu; kullanıcının gerçekte neyi tercih ettiği / hangi olguların öne çıktığı bellek dilimini yönetmek için kullanılır.
- **Cognee** — ECL boru hatları üzerine inşa edilmiş anlamsal bilgi grafiği arka ucu; düz olgular yerine yapılandırılmış, ilişkili bilgileri tutmak için kullanılır.
- **Letta** — Düzenlenebilir bellek blokları ve uyku süresi hesaplamaları içeren durum bilgisi taşıyan ajan çalışma zamanı arka ucu; belleğin canlı ajan durumu olarak kalıcı olması ve boşta kalınan dönemlerde birleştirilmesi gereken durumlarda kullanılır.
- **Graphiti** — İki zamanlı bilgi grafiği arka ucu; olguların ve ilişkilerin yalnızca mevcut değerlerini değil, zaman içindeki değişimlerini de değerlendirmek için kullanılır.
- **PostgreSQL + Neo4j + Redis** — Arka uçların üzerinde çalıştığı gerçek veri depoları; make infra-start komutuyla gerçek entegrasyon testi için ayağa kaldırılır ve test paketi canlı altyapı üzerinde çalıştırılarak taklitler yerine gerçek sistemler sınanır.
- **Prometheus** — Ölçüm ve gözlemlenebilirlik, füzyon boru hattı üzerinden entegre edilmiştir; böylece yönlendirme ve füzyon davranışları üretim ortamında ölçülebilir hale gelir, sistem bir kara kutu olmaktan çıkar.
- **Çok dilli çeviri dikiş noktası** — Ad alanı (helixmemory_) altında tutulan bir metin yüzeyi; gelecekte kullanıcıya yönelik katmanın çekirdek yapıya dokunmadan yerelleştirilebilmesi için hazırda bekletilir.

Durum ve Dürüstlük Notları

- **Durum: beta.** Çalışan SDK; HelixAgent için bellek katmanı olarak geliştirildi.
- **Lisans: Belirlenmedi.** GitHub API aracılığıyla LICENSE dosyası tespit edilemedi — DOĞRULANMAMIŞ / beyan edilmemiş.
- "HelixMemory" görüntü adı, memory deposuna karşılık gelir. README'de belirtilen doğruluk oranları, HelixMemory ölçümleri değil, yukarı akış tedarikçilerinin iddialarıdır ve burada yer almamıştır.

Öncelik seviyesi: Helix-birincil.